Ders 12 Genişletilmiş Notu: Seminer Sunumları: RNN ile Görüntü Sıkıştırma, Denetimsiz Cümle Sıkıştırma, XML Sıkıştırma İncelemesi, Fraktal Görüntü Sıkıştırma ve API Yanıtlarında XML/JSON Sıkıştırma

Ders 12 Genişletilmiş Notu: Seminer Sunumları: RNN ile Görüntü Sıkıştırma, Denetimsiz Cümle Sıkıştırma, XML Sıkıştırma İncelemesi, Fraktal Görüntü Sıkıştırma ve API Yanıtlarında XML/JSON Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Tekrarlayan Sinir Ağlarıyla Tam Çözünürlüklü Görüntü Sıkıştırma
- 3. Sunum 2: Gürültü Giderici Otokodlayıcılarla Denetimsiz Cümle Sıkıştırma
- 4. Sunum 3: XML Sıkıştırma Teknikleri Üzerine Bir İnceleme
- 5. Hocanın Survey Çalışmaları Üzerine Önerisi
- 6. Sunum 4: Fraktal Görüntü Sıkıştırmaya Giriş
- 7. Sunum 5: API Yanıtlarındaki Küçük XML ve JSON Belgelerinin Sıkıştırılması
- Örnek Sorular

1. Dersin Yapısı

Bu ders, önceki iki dersteki gibi öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili yayımlanmış bir makaleyi özetledi. Hoca sunumdan önce konuyu söyledi, sunumdan sonra sorular sordu ve yorum yaptı. Bu derste beş sunum yapıldı: tekrarlayan sinir ağlarıyla görüntü sıkıştırma, gürültü giderici otokodlayıcılarla denetimsiz cümle sıkıştırma, XML sıkıştırma tekniklerini karşılaştıran bir inceleme makalesi, fraktal görüntü sıkıştırma ve web servislerindeki küçük XML/JSON belgeleri için bir sıkıştırma yöntemi.

İlk iki sunum ve fraktal sunumu İngilizce yapıldı. Altyazı bu bölümlerde (özellikle 00:08:00-00:29:00, 00:33:00-00:47:00 ve 01:30:00-01:42:00) çoğu yerde anlamsız tekrara düştü. Bu sunumların anlatımı büyük ölçüde ekrandaki slaytlardan kuruldu. XML sunumları Türkçe yapıldı ve altyazıdan izlenebildi.

2. Sunum 1: Tekrarlayan Sinir Ağlarıyla Tam Çözünürlüklü Görüntü Sıkıştırma

İlk sunumda bir öğrenci, hocanın “RNN’lerle görüntü sıkıştırma” diye tanıttığı “Full Resolution Image Compression with Recurrent Neural Networks” çalışmasını anlattı. Sunuma göre sinir ağlarının görüntü sıkıştırmada başarılı olacağı uzun zamandır düşünülse de tek bir ağın farklı sıkıştırma oranlarında ve farklı görüntü boyutlarında çalışacak biçimde eğitilebileceğine dair az kanıt vardı. Önceki otokodlayıcı tabanlı çalışmalar 64×64 piksellik küçük görüntülerle

sınırlıydı ve çoğu sinir ağı sıkıştırıcısı sabit bir sıkıştırma oranı kullanıyordu; değişken oranlı kodlama için RNN'ler kullanılmıştı.

> **Ek bilgi (derste anlatılmadı):** Bu başlıktaki çalışma büyük olasılıkla Toderici ve arkadaşlarının CVPR 2017'de yayımlanan makalesidir. Aynı grubun daha önceki çalışması 32×32 küçük görüntülerle sınırlıydı; bu makale yöntemi tam çözünürlüklü görüntülere genişletir.

2.1 Genel görüntü sıkıştırma modeli

Sunan öğrenci önce genel görüntü sıkıştırma modelini anlattı (ekrandan, 00:10:00). Model iki temel parçadan oluşur: kodlayıcı ve kod çözücü. Kodlayıcı $f(x, y)$ girdi görüntüsünden sıkıştırılmış bir görüntü üretir; bu görüntü kanaldan iletildikten sonra kod çözücü $f'(x, y)$ çıktı görüntüsünü yeniden kurar. Slayttaki şemada adımlar şöyledir:

- **Kaynak görüntü:** Genelde küçük bloklara bölünür; örneğin JPEG 8×8 piksellik bloklar kullanır.
- **Kaynak kodlayıcı (eşleyici, mapper):** Sinyali frekans uzayına taşır (DFT, DCT, DWT). Pikseller arası artıklığı azaltır. Bu adım tersinirdir, yani kayıpsızdır.
- **Niceleyici (quantizer):** Yüksek frekanslı bileşenleri atar ve psikogörsel artıklığı azaltır. Bu adım kayıplıdır.
- **Entropi kodlayıcı:** Her sembolün olasılığına göre sinyali olabildiğince az bitle temsil eder; en yaygın örneği Huffman kodlamasıdır. Bu adım kayıpsızdır.

2.2 RNN ve LSTM

RNN, zaman boyutunda derin bir yapıya sahip bir derin sinir ağı mimarisidir ve zaman serisi modellemede yaygın kullanılır. Girdi katmanı X, gizli katman S ve çıktı katmanından oluşur; hem o anki girdiyi hem de geçmişte üretilen çıktıyı kullanır. RNN'ler kaybolan gradyan (vanishing gradient) sorunuyla karşılaşır.

LSTM (Long Short-Term Memory), bu sorunu her modüle (hücreye) ek etkileşimler ekleyerek çözer (ekrandan, 00:13:30). Uzun vadeli bağımlılıkları öğrenebilir ve bilgiyi uzun süre hatırlayabilir. Tipik bir LSTM ağı, hücre adı verilen bellek bloklarından oluşur. Bir sonraki hücreye iki durum aktarılır: hücre durumu (cell state) ve gizli durum (hidden state). Hücre durumu verinin ana akış zinciridir ve verinin büyük ölçüde değişmeden ilerlemesini sağlar. Çalışmada incelenen bir başka eleman Associative LSTM'dir; LSTM'i holografik temsil kullanarak genişletir. Çalışmada GRU tabanlı modeller de denenmiştir.

2.3 Önerilen görüntü sıkıştırma modeli

Modelin tek bir yinelemesi kodlayıcı E, ikilileştirici (binarizer) B ve kod çözücü D'den oluşur (ekrandan, 00:16:40). Ağın ağırlıkları yinelemeler arasında paylaşılır, ama durumlar yinelemeden yinelemeye taşınır. Slayttaki gösterimde:

- D_t ve E_t, t yinelemesindeki durumlarıyla birlikte kod çözücü ve kodlayıcıdır.
- b_t ilerlemeli (progressive) ikili temsildir; her yinelemede B ikilileştirilmiş bir bit akışı üretir.
- $\widehat{x}_t$, özgün görüntü x'in ilerlemeli yeniden kurulumudur. "One-shot" yeniden kurmada $\gamma = 0$, toplamsal (additive) yeniden kurmada $\gamma = 1$ alınır.
- r_t, x ile yeniden kurulum $\widehat{x}_t$ arasındaki kalan hatadır (residual).

Slayttaki şemada $32 \times 32 \times 3$ boyutlu kaynak görüntü önce kodlayıcı-ikilileştirici-kod çözücü zincirinden geçer ve yeniden kurulmuş görüntüyle birlikte bir kalan hata (residue) elde edilir. Bu kalan hata bir sonraki yinelemenin girdisidir; ikinci ve üçüncü yinelemeler kalan hatayı tahmin eder (predicted residue). k yineleme sonunda ağ toplam $m \cdot k$ bit üretir. Modeller tamamen evrişimli (fully convolutional) olduğu için m, girdi boyutunun doğrusal bir fonksiyonudur. Şemanın notuna göre sıkıştırılmış çıktı $128 \times M$ bittir (M adım sayısı); her adım 128 bit ekler. Böylece bit hızı (bpp, piksel başına bit) bakımından ilerlemeli bir yöntem elde edilir: daha çok yineleme, daha çok bit ve daha iyi kalite demektir.

2.4 Yeniden kurma çerçevesi

Makalede kod çözücü çıktılarından son görüntüyü kurmanın üç yaklaşımı incelendi (ekrandan, 00:20:00):

1. **One-shot yeniden kurma ($\gamma = 0$):** Her yinelemeden sonra tam görüntü tahmin edilir. Her yeni yineleme kodlayıcının ürettiği daha çok bite erişir ve bu da daha iyi bir yeniden kurma sağlar. Slayta göre bu yaklaşım ağırlık sayısını azaltır.
2. **Toplamsal yeniden kurma ($\gamma = 1$):** Geleneksel görüntü kodlamada daha yaygındır. Her yineleme yalnız önceki yinelemelerden kalan hatayı kurmaya çalışır; son görüntü bütün yinelemelerin çıktılarının toplamıdır.
3. **Kalan hatanın ölçeklendirilmesi (residual scaling):** Hem toplamsal hem one-shot yaklaşımda kalan hata başta büyüktür ve her yinelemede küçülmesi beklenir. Ama kodlayıcı ve kod çözücünün çok geniş bir değer aralığında verimli çalışması zor olabilir. Bu yüzden kalan hata içeriğe göre ölçeklendirilir.

Entropi kodlaması konusunda sunuma göre çıkarım sırasında üretilen kodların entropisi en yüksek değil, çünkü ağ kodundaki entropiyi en yükseğe çıkaracak biçimde açıkça tasarlanmamıştır. Bu yüzden ikili kodun üzerine ayrıca entropi kodlaması uygulanabilir.

2.5 Deneyler ve sonuçlar

Modeller iki eğitim setiyle değerlendirildi: 32×32 görüntülerden oluşan bir veri seti ve web'den rastgele alınmış 6 milyon 1280×720 görüntüden oluşan yüksek entropili veri seti (ekrandan, 00:23:20). Ölçütler şunlardır: kayıplı görüntü sıkıştırma algoritmalarını karşılaştırmada yerleşik bir ölçüt olan MS-SSIM (Multi-Scale Structural Similarity) ve daha yeni olan PSNR-HVS (Peak Signal to Noise Ratio - Human Visual System). MS-SSIM RGB kanallarının her birine ayrı uygulanıp sonuçların ortalaması alınır. Modelleri sıralamak için oran-bozulma (rate-distortion) eğrisinin altında kalan alan (AUC) kullanılır.

32×32 veri setiyle eğitilen modeller:

Model	MS-SSIM sırası	MS-SSIM AUC	PSNR-HVS sırası	PSNR-HVS AUC
GRU (One Shot)	1	1,8098	1	53,15
LSTM (Residual Scaling)	2	1,8091	4	52,36
LSTM (One Shot)	3	1,8062	3	52,57
LSTM (Additive Reconstruction)	4	1,8041	6	52,22
Residual GRU (One Shot)	5	1,8030	2	52,73
Residual GRU (Residual Scaling)	6	1,7983	8	51,25
Associative LSTM (One Shot)	7	1,7980	5	52,33
GRU (Residual Scaling)	8	1,7948	7	51,37
Baseline		1,7225		48,36

Yüksek entropili veri setiyle eğitilen modeller:

Model	MS-SSIM sırası	MS-SSIM AUC	PSNR-HVS sırası	PSNR-HVS AUC
LSTM (One Shot)	1	1,8166	8	48,86

Model	MS-SSIM sırası	MS-SSIM AUC	PSNR-HVS sırası	PSNR-HVS AUC
GRU (One Shot)	2	1,8139	2	53,07
Residual GRU (One Shot)	3	1,8119	1	53,19
Residual GRU (Residual Scaling)	4	1,8076	7	49,61
LSTM (Residual Scaling)	5	1,8000	4	51,25
LSTM (Additive)	6	1,7953	5	50,67
Associative LSTM (One Shot)	7	1,7912	3	52,09
GRU (Residual Scaling)	8	1,8065	6	49,97
Baseline LSTM		1,7408		48,88

Karşılaştırma için JPEG: YCbCr 4:4:4 için 1,7748 ve 51,28; YCbCr 4:2:0 için 1,7998 ve 52,61.

Slayttaki yorumlara göre modeller genelde yüksek entropili veri setiyle eğitildiklerinde 32×32 veri setine göre daha iyi sonuç verir; bu da modelleri "zor" örneklerle eğitmenin önemli olduğunu gösterir. 32×32 veriyle GRU (One Shot) iki ölçütte de en iyisidir; MS-SSIM'de ikinci sırada kalan hata ölçeklemeli LSTM, PSNR-HVS'de ikinci sırada Residual GRU gelir.

Sunan öğrenci Kodak veri setinden görüntülerle görsel karşılaştırma da gösterdi: sol sütunda JPEG 4:2:0, sağ sütunda Residual GRU (One Shot). Yöntemin bit hızları entropi kodlamasından öncedir. İlk satırda 0,25 bit/piksel hedefinde önerilen yöntemin renkleri daha iyi yakaladığı, JPEG'deki gibi renk bloklanması oluşmadığı söylendi.

2.6 Sonuç ve hocanın sorusu

Sonuç bölümüne göre RNN ile sıkıştırma için genel bir mimari sunulmuştur; içeriğe dayalı kalan hata ölçeklendirmesi ve GRU'nun yeni bir varyasyonu, yüksek entropili veri setiyle eğitilen modeller arasında en yüksek PSNR-HVS değerini verir.

Sunumdan sonra hoca, tablolarda geçen "One Shot" ifadesinin ne anlama geldiğini sordu (00:30:49). Öğrencinin cevabı altyazıda anlaşılmıyor.

3. Sunum 2: Gürültü Giderici Otokodlayıcılarla Denetimsiz Cümle Sıkıştırma

İkinci sunumda bir öğrenci "Unsupervised Sentence Compression using Denoising Auto-Encoders" çalışmasını anlattı.

> **Ek bilgi (derste anlatılmadı):** Bu çalışma büyük olasılıkla Févry ve Phang'ın 2018'de CoNLL'de yayımlanan makalesidir.

3.1 Arka plan

Cümle sıkıştırma, metin özetleme probleminin küçültülmüş bir sürümü olarak görülebilir. Metin özetlemede iki geniş yaklaşım vardır:

- **Çıkarımsal (extractive):** Kaynak metinden belirli kelime ya da öbekleri seçip alır.
- **Soyutlayıcı (abstractive):** Kaynak metni, bir insanın yapacağına benzer biçimde sıkıştırılmış olarak yeniden ifade eder.

Derin öğrenmeye dayanan yakın tarihli çalışmalar güçlü dilbilimsel kurallar olmadan eğitilir, ama uzun ve kısa cümle çiftlerinden oluşan büyük derlemlere ihtiyaç duyar. Denetimsiz sinir makine çevirisi üzerine çalışmalar ise uygun bir eğitim düzeniyle, yalnız iki eşlenmemiş tek dilli derlemle iki dil arasında çeviri modeli eğitilebileceğini göstermiştir.

3.2 Modele genel bakış

Model, gürültü giderici otokodlayıcılarla sinir ağı tabanlı metin özetleme tekniklerini cümle sıkıştırmaya uygular ve çıkarımsal özetlemeye odaklanır. Tamamen denetimsizdir: paralel derlem kullanmaz, tek bir derlemle eğitilir. Denetimsiz makine çevirisi hâlâ iki derlem kullanırken bu çalışmada uzun ve kısa cümleler için ayrı derlemler yoktur. Basit bir gürültü giderici otokodlayıcı, gürültü eklenmiş bir girdiden kelimeleri çıkarıp geri kurmayı öğrenir.

Temel model, kodlayıcıda ve kod çözücüde çok katmanlı iki yönlü LSTM içeren standart bir dikkatli (attentional) kodlayıcı-kod çözücüdür; kayıp fonksiyonu negatif log-olabilirliktir (ekrandan, 00:36:40). Denetimsiz eğitim için üç değişiklik yapılır:

- **Additive Noising:** Gürültü giderici otokodlayıcının eğitim düzenini, denetimli cümle sıkıştırmaya daha çok benzeyecek biçimde değiştirerek denetimli bir eğitimi taklit eder.
- **Additive Sampling:** Veri setinden rastgele ek cümleler seçilir ve her birinden yerine koymadan birkaç kelime alt örneklenir.
- **Shuffling:** Elde edilen kelime dizisi karıştırılır. İki tür karıştırma denenir: tam kelime (unigram) karıştırma ve bigram karıştırma.

Slayttaki örnekte referans cümle “Apple Corporation released the new version of the iPhone today.” ve başka bir referans cümle “Barack Obama wins the Democratic Nomination for president.” alınır. İkinci cümleden {Barack, the, Nomination, for} kelimeleri alt örneklenir, birinci cümleyle karıştırılır ve “Barack the new released Apple for of the version Corporation today the iPhone Nomination.” gibi bir girdi oluşur. Kodlayıcı-kod çözücü bundan “Apple released the new iPhone today.” cümlesini kurar; yeniden kurma kaybı referans cümleyle hesaplanır.

3.3 Uzunluk geri sayımı ve cümle gömmesi

İstenen uzunlukta çıktı üretmek için RNN kod çözücüye ek bir uzunluk geri sayımı (length countdown) girdisi verilir. Makalede, istenen zaman adımında dizi sonu belirtecini tahmin etme kaybını büyüten ek bir terim de amaç fonksiyonuna eklenerek denenmiş; ama modelin istenen uzunlukta dizi üretmek için bu ek terime ihtiyaç duymadığı görülmüştür.

Girdi cümle gömmesi bölümünde (ekrandan, 00:40:00) şu noktalar anlatıldı:

- Modele daha kısa bir cümle kurması için yalnız sırasız bir kelime kümesi verilir.
- Özgün cümleyi daha iyi geri kurabilmesi için modele, önceden eğitilmiş bir InferSent modeliyle üretilen özgün cümle gömmesi de verilir.
- InferSent, doğal dil çıkarımı (NLI) görevleriyle eğitilir: uzun bir öncül (premise) metin ve kısa bir hipotez metni verildiğinde model, öncülün hipotezi gerektirdiğini, onunla çeliştiğini ya da ona nötr olduğunu belirler.
- InferSent cümle gömmeleri bu modelin ara çıktısıdır ve her metin dizisinden yakalanan bilgiyi yansıtır.

Sözlük dışı (OOV) kelimeler için numaralı OOV belirteçleri kullanılır. Aynı numaralandırma hedefe de uygulanır; böylece girdide ve çıktıda aynı kelime her zaman aynı numaralı OOV belirtecine, dolayısıyla aynı gömmeye atanır. Çıkarım sırasında çıktıdaki numaralı OOV belirteçleri karşılık gelen kelimelerle değiştirilir. Çıkarımda gürültü uygulanmaz ve uzunluk geri sayımı, referans cümlenin yarısı uzunluğunda çıktı hedefleyecek biçimde ayarlanır. Model PyTorch ile gerçeklenmiştir.

3.4 ROUGE sonuçları

Taban çizgileri, denetimsiz modeller ve denetimli modellerin ROUGE başarımı şöyledir (ekrandan, 00:43:15):

Model	R-1	R-2	R-L	Ort. uzunluk
All text (taban çizgisi)	28,91	10,22	25,08	31,3
F8W (taban çizgisi)	26,90	9,65	25,19	8
2-g shuf (denetimsiz)	27,72	7,55	23,43	15,4
2-g shuf + InferSent (denetimsiz)	28,42	7,82	24,95	15,6
Seq2seq (denetimli)	35,50	15,54	32,45	15,4

Model	R-1	R-2	R-L	Ort. uzunluk
words-lvt2k-1sent (Nallapati vd., 2016)	34,97	17,17	32,70	-

Slayttaki yoruma göre denetimsiz modeller denetimli modellerin gerisinde kalır ve taban çizgileriyle aynı düzeydedir; metin özetleme kıyaslamalarında basit taban çizgileri alışılmadık derecede güçlüdür. Cümle gömmesini kullanan denetimsiz model ROUGE'da biraz daha iyidir.

Ablasyon çalışmasında (ekrandan, 00:44:05) dikkat mekanizması, bigram karıştırma ve cümle gömmesinin etkisi ayrı ayrı ölçüldü:

Model	R-1	R-2	R-L
1-g shuf (dikkatsiz)	23,01	5,51	20,07
2-g shuf (dikkatsiz)	22,36	5,18	19,60
1-g shuf	27,22	7,63	23,55
2-g shuf	27,72	7,55	23,43
1-g shuf + InferSent	28,12	7,75	24,81
2-g shuf + InferSent	28,42	7,82	24,95

Üç bileşenin de ROUGE'a olumlu katkı verdiği ve üçünü birlikte içeren modelin en yüksek puanları aldığı görülür. En büyük sıçrama dikkat mekanizmasının eklenmesindedir (R-1 yaklaşık 23'ten 27'ye çıkar).

3.5 İnsan değerlendirmesi

Altı değerlendiricinin ortalama puanları (ekrandan, 00:44:50):

Model	Dilbilgisi	Anlam
2-g shuf	3,53 (±0,18)	2,53 (±0,16)
1-g shuf + InferSent	2,82 (±0,17)	2,50 (±0,15)
2-g shuf + InferSent	2,87 (±0,16)	2,13 (±0,13)
Seq2seq (denetimli)	3,43 (±0,18)	2,60 (±0,17)
Gerçek özet (ground truth)	4,07 (±0,13)	3,87 (±0,16)

Gerçek özetin anlam puanının görece düşük olması (3,87) şaşırtıcı değildir: Gigaword veri setinde özetler (başlıklar) bazen kaynak metinde olmayan bilgi içerir. Nicel değerlendirmenin insan değerlendirmesiyle iyi örtüşmediği gözlenmiştir: InferSent gömmesini kullanan yöntemler ROUGE puanlarını artırmış, ama insan değerlendirmesinde daha kötü sonuç vermiştir. Sunumda ayrıca girdi (I) ile gerçek özetin (G) yanında model çıktılarını gösteren örnekler verildi; sunan öğrenciye göre bu örneklerde modeller girdideki bilgiyi özetleyebilmektedir.

3.6 Sonuç ve hocanın sorusu

Çalışma, cümle sıkıştırma görevine uyarlanmış gürültü ekleme ve kelime karıştırma içeren gürültü giderici otokodlayıcılarla tamamen denetimsiz bir yaklaşım sunar. Çıktı analizine göre model, Gigaword veri setindeki özetlerin kendine özgü üslubunu yakalayamamaktadır.

Hoca, ROUGE tablosunun bulunduğu 11. slaydı açtırdı (ekrandan, 00:50:00) ve taban çizgisi olarak geçen "F8W" (grafikte "F&W") kısaltmasının açılımını sordu (00:49:23). Öğrencinin cevabı ablasyon çalışmasındaki bileşenleri (dikkat, bigram karıştırma, cümle gömmesi) ve uzunluk geri sayımını tekrar etti; kısaltmanın açılımı verilmedi.

4. Sunum 3: XML Sıkıştırma Teknikleri Üzerine Bir İnceleme

Üçüncü sunumda bir öğrenci, XML sıkıştırma tekniklerini inceleyen ve karşılaştıran bir survey makalesini Türkçe anlattı.

4.1 XML ve sıkıştırma ihtiyacı

XML bir işaretleme dilidir; sistemler arasında veri iletimi ve depolanması gibi birçok alanda kullanılır ve platformdan bağımsızdır. Kendi kendini tanımlar: dışarıdan bakan bir kullanıcı verinin ne anlama geldiğini anlayabilir. Bu, XML'e büyük bir esneklik sağlar. Bir XML belgesinin doğru oluşturulması için iki önemli koşul vardır:

- **Well-formed:** Sözdizimi doğru yazılmış mı?
- **Valid:** Document Type Definition'da (DTD) tanımlanan şemaya uyuyor mu?

XML ayrıştırıcılarının iki türü vardır: SAX ve DOM. Bir XML belgesi element, attribute ve veri değerlerinden oluşur. Etiket, veri ve kapama etiketi aynı nesnedeki her veri için sürekli tekrarlanır; bu da belge boyutunu özgün veriye göre kat kat büyütür. XML sıkıştırma ihtiyacı buradan doğar.

4.2 XML sıkıştırıcılarının sınıflandırılması

Survey, XML sıkıştırıcılarını iki temel özelliğe göre sınıflandırır:

- **XML yapısını bilip bilmemesine göre:**
  - Genel metin sıkıştırıcıları: XML'i normal metin gibi görür. XML hakkında bilgi kullanmadıkları için literatürde "XML-blind" diye adlandırılır.
  - XML-bilinçli sıkıştırıcılar: XML'in kendine özgü yapısından yararlanacak biçimde tasarlanmıştır. Kendi içinde şemaya bağımlı ve şemadan bağımsız diye ikiye ayrılır.
- **Yeteneğine göre:**
  - Sorgulanabilir sıkıştırıcılar: Sıkıştırılmış veri üzerinde sorgulama yapılabilir. Kendi içinde ikiye ayrılır: homomorfik olanlarda belgenin özgün yapısı korunur ve sıkıştırılmış veriye özgün veriyle aynı biçimde erişilir; homomorfik olmayanlarda yapı ve veri parçalar hâlinde ayrı saklanır.
  - Sorgulanamaz sıkıştırıcılar.

4.3 Genel metin sıkıştırıcıları

Bu yöntemlerde XML önce ağaç veri yapısı olarak temsil edilir, sonra bilinen geleneksel metin sıkıştırma yöntemleriyle sıkıştırılır. Üç örnek verildi:

- **gzip:** LZ77 ve Huffman algoritmaları birlikte kullanılarak kayıpsız sıkıştırma yapılır.
- **bzip2:** Öncekine benzer, ama metin sıkıştırılmadan önce Burrows-Wheeler dönüşümü uygulanır. Dönüşüm metni sıralanmış bir biçime getirir ve sıkıştırma başarımını belirgin biçimde artırır; ardından Huffman kodlaması yapılır. gzip'ten daha iyi sıkıştırır, ama yapılan ek işlemler yüzünden daha yavaştır.
- **PPM:** Derste daha önce görülmüştü.

Burrows-Wheeler dönüşümü geçince hoca bu dönüşümün daha önce nerede karşılarına çıktığını sordu ve önceki bir seminerde anlatıldığını hatırlattı.

4.4 Şemadan bağımsız XML-bilinçli sıkıştırıcılar: XMill

Survey'deki şemadan bağımsız örnekler XMill, XWRT, XComp, XMLPPM, SCMPPM ve Exalt'tır. Sunan öğrenci bunlardan birkaçını ayrıntılı anlattı.

XMill önce yapıyı veriden ayırır ve veri değerlerini ağaçtaki yollarına ve türlerine göre homojen kapsayıcılarda gruplar. Belge yapısal bölüm ve veri değerleri bölümü olarak ayrı ayrı sıkıştırılır, en sonda birleştirilir. Slayttaki kurallar (ekrandan, 01:00:00):

- Başlangıç etiketleri ve öznitelik adları sözlükle kodlanır (T1, T2, …).
- Kapama etiketleri "/" belirteciyle değiştirilir.
- Veri değerleri kapsayıcı numaralarıyla (C3, C4, …) değiştirilir.

Slayttaki örnek belge şudur:

```
<Book><Title lang="English">Views</Title>
  <Author>Miller</Author>
  <Author>Tai</Author>
</Book>
```

Atamalar: Book = T1, Title = T2, @lang = T3, Author = T4; “English” C3 kapsayıcısına, “Views” C4 kapsayıcısına, “Miller” ve “Tai” ise aynı yolda oldukları için aynı C5 kapsayıcısına girer. Yapı akışı öneklerle şöyle okunur:

```
T1
  T2
    T3 C3 /
    C4 /
  T4 C5 /
  T4 C5 /
```

XMill’in genel işleyişi (ekrandan, 01:02:00): XML girdisi SAX ayrıştırıcıdan geçer, yol işlemcisi (path processor) yapıyı ve verileri ayırır. Komut satırında verilen kapsayıcı ifadeleri (örneğin `-p//apache:host=>IP`) hangi yolların hangi kapsayıcıya gideceğini belirler. Yapı kapsayıcısı (`T1 T2 C1 / T3 C2 / ...`) ve veri kapsayıcıları bellekte tutulur, her biri ayrı ayrı gzip ile sıkıştırılır ve birleştirilerek saklanır. Slayttaki Apache günlüğü örneğinde yapı ve veri birlikte gzip ile sıkıştırıldığında 1,75 MB, veri iki ayrı kapsayıcıya (IP adresleri ve istek satırları) bölünüp sıkıştırıldığında 1,33 MB elde edilir.

4.5 Diğer XML-bilinçli sıkıştırıcılar

- **XWRT:** XMill’e benzer bir yapıdadır, ama Word Replacing Transform adlı sözlüğe dayalı bir dönüşüm kullanır. Arkasından gzip, LZMA ya da PPM uygulanabilir.
- **XComp:** XML belgesi önce ayrıştırılır, bileşenler düzenlenerek sıkıştırma motoruna iletilir.
- **XMLPPM:** Çoklu hiyerarşik (multiplexed hierarchical) PPM modeli kullanır.
- **SCMPPM:** PPM modellerinin daha büyük bir kümesini kullanır.
- **Exalt:** Dilbilgisel bir yaklaşımdır. Bağlamdan bağımsız bir dilbilgisi dinamik olarak oluşturulur, ardından aritmetik kodlama yapılır.
- **AXECHOP:** Dilbilgisi tabanlı bir yaklaşımdır. Belge yapısal ve veri bölümlerine ayrılır; özgün yapıyı koruyan bir byte işaretleme şeması kullanılır. Yapı dizgisi MPM (Multi-Level Pattern Matching) algoritmasıyla dilbilgisine çevrilir, sonra Burrows-Wheeler sıralamasından geçirilip sıkıştırılır. Çözme bunun tersidir: önce MPM’den geri dönülür, sonra BWT’nin tersi uygulanır ve veri elde edilir.

Şemaya bağımlı yöntemler (altyazıda “DTD PPM” ve benzerleri) pek kullanılmaz, çünkü hem sıkıştırırken hem çözerken şemaya ihtiyaç vardır; alıcının ve göndericinin XML şemasını bilmesi gerekir.

4.6 Sorgulanabilir XML sıkıştırıcıları

Sorgulanabilir sıkıştırıcılarda XML tamamen açılmadan sıkıştırılmış veri üzerinde sorgulama yapılabilir: tam ya da kısmi eşleşme bulunabilir, aralık sorguları yapılabilir, veri açılmadan güncellenebilir ve doğrulama yapılabilir. Homomorfik örnekler XGrind, XPRESS ve QXT; homomorfik olmayan örnekler XSeq, XCQ, TREECHOP, XQzip, XBzip, ISX, XCpaqs ve XQueC’tir.

XGrind: Bağlamdan bağımsız sıkıştırma kullanır. Etiketler XMill’deki gibi kodlanır, ama XMill’den farklı olarak özniteliklere de ayrı bir kod atanır. Kapama etiketleri “/” ile değiştirilir. Her yaprak (veri değeri) Huffman sıkıştırıcısından geçirilir. Sorgu motoru sıkıştırılmış veri üzerinde çalışır; bir sözlük ve bir ayrıştırıcı içerir. Sorgu önce sıkıştırılır, sonra sıkıştırılmış veri üzerinde yolun eşleşip eşleşmediği ve yüklemin sağlanıp sağlanmadığı kontrol edilerek sonuç çıkarılır.

TREECHOP: Belgelerin ağ üzerinden iletildiği XML mesajlaşma senaryolarına yönelik bir yöntemdir; kodlama ve sorgulama XPath standardına dayanır. Sıkıştırma stratejisi (ekrandan, 01:10:00):

1. SAX ayrıştırması yapılır.
2. Belge ağacı oluşturulur.
3. Yaprak olmayan her ağaç düğümüne bir ikili kod sözcüğü atanır.
4. Ağaç kodlaması sıkıştırma akışına yazılır.

Kodlamanın üç önemli özelliği vardır:

- Her ağaç düğümü atasının kodunu önek olarak alır.
- İki düğüm, ancak ve ancak kökten aşağı izlenen XPath konumları aynıysa aynı kod sözcüğünü paylaşır.
- Özgün belgenin yapısı sıkıştırma boyunca korunur; bu yüzden açmadan sorgulama yapılabilir.

Sunan öğrencinin örneğinde class'tan türeyen etiketlerin (note, name, students gibi) kodları class'ın kodu olan 00 ile başlar. Students altındaki iki note düğümünün kodunun aynı olması, ikisinin aynı yolda olduğunu gösterir; bu özellik sorgulamada kullanılır. XPath ile sorgulamada önce yolun karşılık geldiği ikili kod adım adım bulunur, sonra bu kodun altındaki yapraklardaki bütün veriler çekilir.

4.7 Hocanın yorumu

Sunumun ortasında hoca araya girdi (01:10:35) ve survey'de çok sayıda yöntem bulunduğunu, anlatılacak yöntemlerin başarısı yüksek olanlardan seçilmesi gerektiğini söyledi. Sunan öğrenci, görsel olarak anlatılabilecek yöntemleri seçtiğini, diğerlerinin daha soyut kaldığını belirtti.

4.8 Deney düzeni ve sonuçlar

Survey'de internetten erişilen 27 XML belgesi kullanılmıştır; bunların arasında Wikipedia verisi ve yapay olarak üretilmiş belgeler vardır. Belgelerin özellikleri karşılaştırılırken yapının veriye oranı önemlidir, çünkü şema çok büyük, içindeki veri az olabilir. Yapının etkisini ortaya çıkarmak için yazarlar kendi yazdıkları bir araçla 27 belgeden yalnız yapı içeren (verisi olmayan) 30 yapısal belge üretmiştir. Deneyler yapısal ve özgün belgeler üzerinde ayrı ayrı yapılmıştır.

Sonuçlar:

- Yapısal belgelerde en yüksek sıkıştırma oranını PPM vermiştir; özgün belgelerde ise en iyi başarımı XMill-gzip vermiştir. Yani sıkıştırma oranı hem belgenin türüne hem kullanılan yönteme bağlıdır.
- Ortalama sıkıştırma oranlarında, özgün veride gzip iyi sonuç verirken yapısal (verisi olmayan) belgelerde SCMPPM daha iyidir.
- Sıkıştırma oranı, sıkıştırma süresi ve açma süresi grafiğinde (ekrandan, 01:16:05; Bzip2, Gzip, PPM, XMillBzip2, XMillGzip, XMillPPM, XMLPPM, XWRT, SCMPPM) SCMPPM'in sıkıştırma süresi makuldür, ama açma süresi çok uzundur; XMLPPM ve XMillPPM'de de açma süresi uzundur. Sıkıştırma oranı bakımından SCMPPM öne çıkar.

İnceleme makalesi olduğu için çalışmanın amacı, bu alanda ileride çalışma yapacaklara yol göstermektir.

5. Hocanın Survey Çalışmaları Üzerine Önerisi

XML sunumundan sonra hoca, doktora yapan öğrencilere bir öneride bulundu. Çalışma konularını belirlediklerinde, en geç tez önerisine girmeden önce ya da ilk izlemede, bu sunumdaki gibi bir survey yapmalarını önerdi: 2021'den başlayıp geriye doğru bu alanda kimin ne yaptığı, hangi veri setini ve yöntemi kullandığı, nasıl bir sonuç aldığı çıkarılmalıdır. Çalışma ayrıntılı yapılır, yöntemler gruplar hâlinde sunulabilirse ve son 3-4 yıl içinde aynı konuda bir survey yoksa bunun yayın olarak gönderilmesi gerekir, çünkü survey'ler çok atıf alır.

Hoca survey'in amacını şöyle açıkladı: bir konuda çalışacak kişi onlarca makale okumak yerine, birinin bu makaleleri okuyup çıkardığı özeti okur. Ama aynı konuda yakın tarihli bir survey varsa (örneğin yılın başında yayımlanmış bir survey varken yılın sonunda aynı konuda yenisi hazırlanırsa) iki çalışma birebir aynı olur ve kabul almak zordur. Yeni yayınlar ekleyebilmek ve yenilik katabilmek için son 2-3 yılda benzer bir survey yapılmamış olmalıdır.

> **Hocanın vurgusu:** Survey hazırlamadan önce o konuda yakın tarihli bir survey olup olmadığı ve tarihleri mutlaka kontrol edilmelidir.

6. Sunum 4: Fraktal Görüntü Sıkıştırmaya Giriş

Dördüncü sunumda bir öğrenci, daha yüksek sıkıştırma başarımı için görüntülere uygulanabilecek kayıplı yöntemlerden biri olarak fraktal görüntü sıkıştırmaya giriş niteliğindeki bir çalışmayı İngilizce anlattı. Altyazı bu sunumun büyük bölümünde bozuk olduğu için anlatım slaytlardan ve okunabilen cümlelerden kuruldu.

6.1 Fraktal görüntü ve afin dönüşümler

Fraktal görüntü, afin dönüşüm adı verilen özel dönüşümlerin art arda uygulanmasıyla üretilir; bu dönüşümler eğme (skew), germe (stretch), döndürme (rotate) ve ölçekleme (scale) yapabilir (ekrandan, 01:33:00). Bir afin dönüşüm şöyle yazılır:

$$w_i \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_i & b_i \\ c_i & d_i \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e_i \\ f_i \end{bmatrix}$$

Afin dönüşüm görüntünün geometrik yapısını değiştirir: doğruların paralelliğini korur, ama uzunlukları ve açıları korumaz. Her w_i dönüşümü yalnız 6 sayıyla $(a_i, b_i, c_i, d_i, e_i, f_i)$ tanımlanır; bunları saklamak çok bellek gerektirmez. Yöntem bir görüntüyü, fotokopi makinesi benzetmesine çok benzeyen bir dönüşüm topluluğu olarak kodlar.

> **Ek bilgi (derste anlatılmadı):** Fotokopi makinesi benzetmesinde makine görüntüyü küçültülmüş kopyalar hâlinde yeniden dizer ve çıktısı tekrar girdisi yapılır. Dönüşümler daraltan (contractive) olduğu için, hangi görüntüden başlanırsa başlansın yinelemeler aynı sabit görüntüye yakınsar. Fraktal kod çözmenin birkaç yinelemede yakınsaması bu özelliğe dayanır.

6.2 Kodlama

Sunan öğrencinin okunabilen cümlelerine göre farklı afin dönüşümler farklı veri blokları üretir. İnsan gözünün görüntüdeki küçük değişikliklere duyarsız olmasından yararlanılır; fraktal görüntü kodlamanın kayıplı olmasının bir nedeni budur. Kodlamada aralık (range) görüntüsündeki her bloğa çeşitli dönüşümler uygulanarak, önceki aşamada elde edilen alan (domain) görüntüsünde benzer bir blok aranır. Eşleşen bloğun birebir aynı değil yalnız benzer olması, yöntemin kayıplı olmasının ikinci nedenidir. Kodlamanın sonunda yalnız bloklar ve özgün görüntüyü elde etmek için kullanılacak afin dönüşümler saklanır. Bir blok birden çok (iki, üç ya da dört) blok üretebilir.

6.3 Kod çözme

Kod çözme denklemi iki parçadan oluşur (ekrandan, 01:40:00):

$$\eta = \Gamma(\Omega)\, \Psi(\Omega)$$

- $\Gamma(\Omega)$, bir alan görüntüsü üretmek için Ω görüntüsüne uygulanan alt örnekleme ve alçak geçiren süzgeçtir.
- $\Psi(\Omega)$, fraktal kodda kaydedilmiş, alan görüntüsündeki alan bloklarından aralık görüntüsündeki aralık bloklarına eşlemelerle tanımlanan dönüşümlerin topluluğudur.
- Ω_n, 7'den az yinelemede özgün görüntünün iyi bir yaklaşımına yakınsar.

6.4 Sonuçlar

Çalışmada fraktal sıkıştırmanın iki uygulaması karşılaştırıldı: 2×2 bloklu ve 4×4 bloklu. Lena görüntüsü (128×128) ile sonuçlar şöyledir (ekrandan, 01:41:20):

	Yöntem I	Yöntem II
Blok boyutu	2×2	4×4
Yineleme sayısı	6	6
Kodlama süresi	10 dk	55 s

	Yöntem I	Yöntem II
Kod çözme süresi	45 s (?)	28 s
Kod kitabı boyutu	16384 byte	6144 byte
SNR	27 dB	21 dB
Tepe hata	115	95
Bit hızı	1 byte/piksel	0,375 byte/piksel
Referans görüntü	kare	kare

2×2 blok daha yüksek SNR verir, ama kodlaması çok daha uzun sürer ve daha çok bit harcar. Bit hızı, kod kitabı boyutunun piksel sayısına bölümüdür: $128 \times 128 = 16384$ piksel için $16384/16384 = 1$ byte/piksel ve $6144/16384 = 0{,}375$ byte/piksel (ara adım eklendi; python ile doğrulandı).

Sunumdan sonra fraktallarla ilgili soru soran olmadı.

7. Sunum 5: API Yanıtlarındaki Küçük XML ve JSON Belgelerinin Sıkıştırılması

Son sunumda bir öğrenci, hocanın "XML ve JSON compression, API responses" diye tanıttığı bir çalışmayı Türkçe anlattı. Bir önceki XML sunumundan farkı, web API'lerinde kullanılan küçük boyutlu XML ve JSON belgelerinin sıkıştırılmasına odaklanmasıdır.

7.1 XML ve JSON

Günümüzde internetteki projelerin büyük çoğunluğu web servislerini kullanır; JSON ve XML bu servislerin ortak bir dille iletişim kurmasını sağlar. XML çok fazla gereksiz veri içerir, çünkü aktarılacak her özellik için açma etiketi, veri ve kapama etiketi gönderilir. JSON'da kapama etiketi yoktur; bu yüzden JSON belgeleri genelde biraz daha küçüktür. JSON'da özellikleri tanımlayan etiketlere anahtar (key), XML'de element ya da attribute denir; ikisinin kullanımı arasında temel bir fark yoktur (örnekteki City özniteliği ayrı bir element olarak da yazılabilirdi).

Slayttaki örnek (ekrandan, 01:48:10):

```
JSON:
{
  "Havaalanı": {
    "Varış-Zamanı": "16:02:30",
    "Kalkış-Zamanı": "16:22:30",
    "Tarih": "2019-12-24",
    "Kapı": [6, 7]
  }
}

XML:
<Havaalanı City="Edmonton">
  <Varış-Zamanı>16:02:30</Varış-Zamanı>
  <Kalkış-Zamanı>16:22:30</Kalkış-Zamanı>
  <Tarih>2019-12-24</Tarih>
  <Kapı>6</Kapı>
  <Kapı>7</Kapı>
</Havaalanı>
```

Açıklayıcı etiketler servise yapılan her istekte ve yanıtta tekrarlanır. Bu tekrarlar belgeyi kendini tanımlayıcı (self-descriptive) yapar, ama gereksiz veriyle doldurur. Sunan öğrenciye göre makaledeki bir uçuş yanıtı örneğinde veri sıralı olarak gönderilse 55 byte tutar, ama kendini tanımlamaz ve anlamlandırılması zor olur. Aynı veri JSON olarak 154 byte, XML olarak 169 byte tutar; yani boyut neredeyse 3 katına çıkar ($154/55 = 2{,}8$, $169/55 \approx 3{,}07$; python ile doğrulandı). Çalışmanın başlığında JSON da geçse de anlatım XML üzerinden yapıldı.

Bu noktada hoca bir soru sordu (01:48:41): XML'de çok fazla gereksiz kısım var, ama JSON çok sade duruyor; JSON'da neyi sıkıştırıyorlar? Sunan öğrenciye göre aynı yöntem JSON'a da uygulanabilir: anahtarlardan bir sözlük oluşturulur ve anahtarlar kısaltılır; verilerde pek bir şey yapılmaz.

7.2 Mevcut çalışmalar: XMill ve SMCA

Problemi anlamak ve bir araştırma sorusu önermek için iki önemli çalışma incelendi: XMill ve ondan türetilmiş, daha gelişmiş bir algoritma olan SMCA. Ortak özellikleri (ekrandan, 01:50:00):

1. Her iki teknik de XML'in veri ve yapı bölümlerini birbirinden bağımsız işler.
2. Her iki teknik de XML'in yapı bölümlerine belgedeki konumlarına bağlı olarak sayılar atar ve bir etiket sözlüğü oluşturur. Örnekte yukarıdan başlayarak Havaalanı 1, şehir 2 numarayı alır; ardından varış zamanı, kalkış zamanı, tarih ve kapı gelir.
3. Her veri öğesi için bir yol çıkarılır: en tepeden söz konusu veri öğesinin üst etiketine kadar olan etiket adları dizisi. Örneğin 16:02:30 için yol Havaalanı/Varış-Zamanı, 6 için Havaalanı/Kapı'dır. Yollar etiketler için değil, veriler için çıkarılır.
4. Her yeni yol için yeni bir kapsayıcı oluşturulur; ortak yolu paylaşan bütün veriler aynı kapsayıcıya eklenir. Temel dayanak, tekrar eden verileri bir kapsayıcıda bzip gibi bir sıkıştırıcıyla sıkıştırmanın iyi sonuç vermesidir.

Sorunlar:

- Bazı XML belge türlerinde her biri az sayıda veri öğesi içeren çok sayıda kapsayıcı oluşur. API istek ve yanıtları kısa olma eğilimindedir ve az tekrar içerir; bu da sıkıştırma oranını olumsuz etkiler.
- Etiketlere sırasıyla sayı atanması, sık olmayan yapısal öğelere küçük, sık öğelere büyük sayılar verilmesine yol açabilir. Çözüm olarak en sık öğeye en küçük sayının atanması önerilir.
- İki yöntem de sıkıştırılmış biçimde XML yapısını korumaz; bu yüzden sıkıştırılmış belge sorgulanamaz, arama için önce açmak gerekir. Önerilen yöntem de böyle olduğu için sunan öğrenci bunu bir dezavantaj olarak görmedi.

7.3 Önerilen yöntem: yapılandırma aşaması

Önerilen yöntem iki aşamalıdır: yapılandırma aşaması ve üç adımlı sıkıştırma aşaması. Yapılandırma aşaması, sıkıştırmada kullanılacak iki sözlük oluşturur: etiket tablosu ve sembol tablosu.

- **Etiket tablosu:** XMill ve SMCA'dan farklı olarak etiketlere sıklıklarına göre numara verilir. Örnekte en çok tekrar eden etiket Kapı olduğu için 1 numarayı alır; diğer etiketler yukarıdan aşağı numaralanır.
- **Sembol tablosu:** XML belgelerinin ana dili İngilizce kabul edilir. İngilizce karakterler ASCII tablosunda 32 ile 127 arasındadır. Makaleye göre yalnız bu karakterleri kodlamak yeterlidir. Her karakter, ASCII değeri eksi 31 olarak hesaplanan iki basamaklı bir sayıyla eşlenir; böylece karakterler 1'den $127 - 31 = 96$'ya kadar numara alır.

Sunan öğrenci, internet içeriğinin yaklaşık %90'ının İngilizce olduğunu, ikinci sırada yaklaşık %5 ile Rusçanın, okuduğu makaleye göre üçüncü sırada Türkçenin geldiğini (başka kaynaklarda İspanyolca) söyledi.

> **Ders sonrası not:** Sunumda internet içeriğinin %90'ının İngilizce olduğu söylendi. Bu oran büyük olasılıkla abartılıdır; W3Techs gibi web içerik istatistiklerinde 2021 yılı için İngilizcenin payı büyük olasılıkla %60 civarında verilir. Yöntemin İngilizce karakter varsayımı bu orandan bağımsız olarak geçerlidir.

7.4 Önerilen yöntem: sıkıştırma aşaması

Sıkıştırma üç adımdan oluşur:

1. **Karakter çevirme:** Belge, her biri 4 bitle temsil edilebilen sembollerden oluşan XMLN biçimine çevrilir. Semboller rakamlar ile T (etiket), A (öznitelik) ve 0 (kapama etiketi) işaretleridir.
2. **Paketleme:** 4 bitle kodlanan iki sembol birleştirilerek 8 bitlik bir byte'a yerleştirilir. Sonuçta XMLS adı verilen sıkıştırılmış bir ikili dosya oluşur.
3. **Arka plan sıkıştırıcı:** XMLS dosyası bir düz metin sıkıştırıcısıyla yeniden sıkıştırılır; bu çalışmada PPM kullanılmıştır.

Slayttaki XMLN örneği (ekrandan, 01:54:30):

```
T2 A4 1314150616070616 T3 2526272829273028 0
T6 2526272929273028 0 T5
29282534232529232935
0 T1 23 0 T1 24 0 0
```

Sunan öğrencinin anlatımıyla okunuşu şöyledir. T2 bir etikettir; etiket tablosunda 2 Havaalanı'dır. Ardından A4 gelir: 4 numaralı öznitelik, yani City (şehir). Sonra bu özniteliğin verisi (Edmonton) gelir. Öznitelik olduğu için kapama etiketine gerek yoktur, bu yüzden 0 konmaz. Sonra bir alt etiket ve verisi gelir; bu etiket kapatılmalıdır, yoksa yazılacak yeni etiket onun çocuğu sayılır. Böylece her alt etiket verisinden sonra 0 ile kapatılır ve en sonda dış etiket de kapatılır.

Örnekteki sayı dizileri kendi içinde tutarlıdır (python ile karşılaştırıldı): "Edmonton" için E = 13, d = 14, m = 15, o = 06, n = 16, t = 07 alınırsa `1314150616070616` elde edilir; iki "o" ve iki "n" aynı kodu alır. Saatler ve tarih için 1 = 25, 6 = 26, ":" = 27, 0 = 28, 2 = 29, 3 = 30, 9 = 34, "-" = 23, 4 = 35 alınırsa 16:02:30, 16:22:30 ve 2019-12-24 slayttaki dizileri verir.

> **Ders sonrası not:** Slayttaki kodlar sunumda anlatılan "ASCII değeri eksi 31" kuralıyla örtüşmüyor: bu kurala göre E için $69-31=38$, 1 için $49-31=18$ beklenirdi; slaytta 13 ve 25 kullanılmış. Ayrıca Kapı değerleri 6 ve 7 slaytta 23 ve 24 olarak yazılmış; oysa aynı örnekte 6 = 26 ve "-" = 23'tür. Örnek büyük olasılıkla temsilîdir ya da slaytta hata vardır.

7.5 Sonuçlar ve dezavantajlar

XML boyutu ile sıkıştırma oranının karşılaştırıldığı grafikte (ekrandan, 01:56:10) önerilen yöntem (kırmızı) küçük belgelerde SMCA'dan (mavi) belirgin biçimde daha yüksek sıkıştırma oranı verir; boyut büyüdükçe iki yöntemin oranları 20-23 civarında birleşir. Sunan öğrenciye göre API çağrıları genelde 500 KB'ın altındadır; yöntem 1 MB'a kadar üstünlüğünü sürdürür, sonra SMCA ile hemen hemen aynı başarımı verir. Küçük veri setlerinde önerilen yöntem XMill ve SMCA'dan daha iyidir; veri büyüdükçe avantajı kaybolur.

Makalede belirtilen dezavantajlar:

- XML'deki karakter sayısı arttıkça sembol tablosundaki numaralar büyür. Örneğin İngilizce dışında başka diller de desteklenmek istenirse iki basamaklı sayılar yetmez, üç basamaklı sayılar gerekir. Bu da sıkıştırma oranını düşürür; çok sayıda karakter gereken durumlarda yöntemin etkinliği şüphelidir.
- XML içinde ikili veri saklanıyorsa, normalde 1 bitle kodlanabilecek veri 4 bitle kodlanır. Sunan öğrenciye göre bu durumda metin sıkıştırıcısı uygulandıktan sonra elde edilen 1,33 katlık kazanç hemen hemen sıfırlanır; dosya başladığı yere döner.
- Küçük XML belgelerinde önerilen yöntemin sıkıştırma oranı SMCA'dan iyidir; ama özellikle 2 MB'tan büyük belgelerde çoğunlukla SMCA'ya eşittir.

Sunumdan sonra hoca anlatımın hızlı ve akıcı olduğunu söyledi; başka soru gelmedi.

Örnek Sorular

Soru 1 (derste sözlü, 00:30:49). RNN ile görüntü sıkıştırma çalışmasının tablolarında geçen "One Shot" ne anlama gelir? Toplamsal (additive) yeniden kurmadan farkı nedir?

Çözüm (slaytta verildi, ekrandan 00:20:00): One-shot yeniden kurmada ($\gamma = 0$) her yinelemeden sonra kod çözücü tam görüntüyü tahmin eder; her yeni yineleme kodlayıcının ürettiği daha çok bite eriştiği için yeniden kurma iyileşir. Toplamsal yeniden kurmada ($\gamma = 1$) her yineleme

yalnız önceki yinelemelerden kalan hatayı kurar ve son görüntü bütün yineleme çıktılarının toplamıdır. Öğrencinin sözlü cevabı altyazıda anlaşılmıyor.

Soru 2 (derste sözlü, 00:49:23; ekrandan, 00:50:00). Cümle sıkıştırma çalışmasının ROUGE tablosunda taban çizgisi olarak geçen "F8W" (grafikte "F&W") neyin kısaltmasıdır?

> **Çözüm (derste verilmedi):** Derste açılımı verilmedi. Tablodaki ortalama uzunluğu tam 8 kelime olduğu için büyük olasılıkla "First 8 Words", yani girdinin ilk 8 kelimesini özet olarak alan basit bir taban çizgisidir. "All text" taban çizgisi ise girdinin tamamını alır (ortalama uzunluk 31,3).

Soru 3 (derste sözlü, 00:57:15). bzip2'de sıkıştırmadan önce uygulanan dönüşüm hangisidir ve derste nerede karşımıza çıkmıştı?

Çözüm (derste verildi): Burrows-Wheeler dönüşümüdür. Metni sıralanmış bir biçime getirir ve ardından gelen sıkıştırmanın (bzip2'de Huffman) başarımını artırır. Hoca bu dönüşümün önceki bir seminer sunumunda anlatıldığını hatırlattı.

Soru 4 (derste sözlü, 01:48:41). XML'de çok fazla gereksiz kısım vardır, ama JSON zaten sade görünür. JSON belgesinde neyi sıkıştırırız?

Çözüm (derste verildi): Aynı yöntem JSON'a da uygulanabilir. Anahtarlardan (key) bir sözlük oluşturulur ve anahtarlar kısaltılır (numaralanır); veri değerlerinde pek bir şey yapılmaz.

Soru 5 (ekrandan, 01:00:00). XMill, `<Book><Title lang="English">Views</Title><Author>Miller</Autho` belgesinin yapısını nasıl kodlar?

Çözüm (slaytta verildi): Book = T1, Title = T2, @lang = T3, Author = T4 olur. Veri değerleri kapsayıcı numaralarıyla değiştirilir: English → C3, Views → C4, Miller ve Tai aynı yolda oldukları için ikisi de C5. Kapama etiketleri "/" olur. Yapı akışı: `T1 T2 T3 C3 / C4 / T4 C5 / T4 C5 /`. Veri değerleri ayrı kapsayıcılarda saklanıp her kapsayıcı ayrı sıkıştırılır.

Soru 6 (ekrandan, 01:41:20). 128×128 Lena görüntüsü için fraktal sıkıştırmada kod kitabı 2×2 blokla 16384 byte, 4×4 blokla 6144 byte tutuyor. Piksel başına bit hızları nedir?

> **Çözüm (derste verilmedi):** Piksel sayısı $128 \times 128 = 16384$'tür. 2×2 blokta $16384/16384 = 1$ byte/piksel (8 bit/piksel), 4×4 blokta $6144/16384 = 0{,}375$ byte/piksel (3 bit/piksel). Slayttaki bit hızlarıyla aynıdır (python ile doğrulandı).

Soru 7 (ekrandan, 01:48:10). Örnekteki uçuş verisi sıralı gönderilirse 55 byte, JSON olarak 154 byte, XML olarak 169 byte tutuyor. JSON ve XML gösterimi veriyi kaç kat büyütür?

> **Çözüm (derste verilmedi):** JSON: $154/55 = 2{,}8$ kat; XML: $169/55 \approx 3{,}07$ kat. Sunumda "neredeyse 3 katı" denmiştir (python ile doğrulandı).

Soru 8 (ekrandan, 01:54:30). XMLN dizisi `T2 A4 1314150616070616 T3 2526272829273028 0 ...` ile başlıyor. T, A ve 0 neyi gösterir ve A4'ten sonra neden 0 gelmez?

Çözüm (derste verildi): T etiketi, A özniteliği, 0 kapama etiketini gösterir. T2 etiket tablosundaki Havaalanı, A4 City özniteliğidir; ardından özniteliğin verisi (Edmonton) gelir. Öznitelikler kapama etiketi gerektirmediği için A4'ün verisinden sonra 0 konmaz. T3 ise bir alt etikettir (Varış-Zamanı 16:02:30); sonraki etiketin onun çocuğu sayılmaması için verisinden sonra 0 ile kapatılır.

Dersin Toparlaması

Bu derste beş seminer sunumuyla veri sıkıştırmanın görüntü, metin ve yapılandırılmış veri alanlarındaki uygulamaları görüldü. İlk sunum, derste anlatılan dönüşüm-niceleme-entropi kodlama hattını sinir ağlarıyla kurdu: RNN tabanlı kodlayıcı-ikilileştirici-kod çözücü, her yinelemede kalan hatayı kodlayarak bit hızını adım adım artırır. İkinci sunum "sıkıştırma"yı cümle düzeyinde ele aldı; gürültü giderici otokodlayıcı paralel veri olmadan cümleyi kısaltmayı öğrenir, ama ROUGE ile insan değerlendirmesinin her zaman örtüşmediği görüldü. Fraktal sıkıştırma sunumu ise görüntüyü afin dönüşümlerle kodlayan, blok boyutuyla kalite ve hız arasında ödünleşim sunan kayıplı bir yöntemi tanıttı.

XML üzerine iki sunum birbirini tamamladı. İlki, XML sıkıştırıcılarını XML'i bilip bilmemelerine ve sorgulanabilirliklerine göre sınıflandıran bir inceleme makalesiydi; XMill'in yapı-veri

ayrımı ve kapsayıcı fikri, gzip, bzip2 (Burrows-Wheeler) ve PPM gibi derste görülen genel sıkıştırıcıların üzerine kurulur. İkincisi, bu fikirlerin API yanıtları gibi küçük belgelerde neden zayıfladığını ve sıklığa göre numaralandırma, 4 bitlik semboller ve PPM ile nasıl iyileştirilebileceğini gösterdi. Hoca da bu vesileyle doktora öğrencilerine kendi konularında survey hazırlamanın değerini ve yakın tarihli survey kontrolünün önemini anlattı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.